FitBuddy

Coding and Solution Guide

Implementation blueprint based on the supplied FitBuddy project brief

What the app should do

Collect user details, goal, and workout intensity; generate a 7-day workout plan and a concise supporting tip; save the original plan; accept feedback; generate and store an updated plan; display both versions in an authorized admin view.

Suggested code layout

`app/main.py`	Create FastAPI app and register routes.
`app/routes.py`	Handle pages, forms, validation, and responses.
`app/gemini_generator.py`	Generate structured workout plans.
`app/gemini_flash_generator.py`	Generate short nutrition/recovery tips.
`app/updated_plan.py`	Revise plans using feedback.
`app/database.py`	Save and retrieve users and plans.
`app/models.py`	Database models and input schemas.
`templates/`	index, result, and admin pages.

Build in this order

1. Create the environment and project folders; install the brief’s dependencies.
2. Define user and plan database models; create and verify tables.
3. Build the input page and validate required fields and allowed intensity values.
4. Implement workout and tip generation as separate service functions.
5. Add routes to save and display the generated results.
6. Add feedback revision and preserve original and updated plans separately.
7. Add protected admin view; document setup and test the user flows.

Main user flow

Input form -> validate -> AI services -> save records -> render results
Feedback form -> load original plan -> AI revision -> save updated plan -> render

FitBuddy

Core Logic and Implementation Notes

Plan generation logic

```
def create_plan(user):
   profile = validate(user)
   plan = generate_workout_plan(profile)
   tip = generate_nutrition_tip(profile.goal)
   save_user_and_original_plan(profile, plan)
   return plan, tip
```

Use the function flow as a design guide; connect each step to your app’s actual FastAPI routes, Gemini SDK calls, and SQLAlchemy session management.

Feedback update logic

```
def revise_plan(user_id, feedback):
   original = get_original_plan(user_id)
   updated = update_workout_plan(original, feedback)
   save_updated_plan(user_id, feedback, updated)
   return updated
```

Implementation guidance

Keep AI prompts and API setup inside service modules, not duplicated in route code.
Validate age, weight, goal, intensity, and user ID before using or storing values.
Handle AI timeouts, empty responses, and database errors with clear user messages.
Keep the original plan unchanged; store feedback and each revision distinctly.
Read GOOGLE_API_KEY from environment configuration; exclude .env from Git.
Do not expose user records through the admin route without access checks.

Completion checklist

A valid user can generate a plan and tip; the results page displays them clearly.
Feedback revises a plan and preserves the original; records persist after restart.
Invalid input, unavailable AI service, and missing users produce helpful errors.
Admin access is restricted; README explains setup, key configuration, and routes.